

UNIVERSIDAD · FACULTAD DE INGENIERÍA

Lenguajes de Programación I

Lenguajes de Programación en C y C++

Unidad 1: Fundamentos, Paradigmas y Programación Estructurada

Unidad 2: Complejidad, Archivos, Estructuras y Lenguajes Modernos

GILMA COAQUIRA SALLUCA

2026

Compilación de teoría, conceptos y ejemplos prácticos

Lenguajes de Programación en C y C++

Gilma Coaquira Salluca

Universidad — Facultad de Ingeniería
Lenguajes de Programación I

© 2026

*Todos los derechos reservados.
Este libro ha sido preparado con fines académicos.*

Prefacio

Este libro reúne los contenidos completos del curso de **Lenguajes de Programación I**, organizado en dos unidades progresivas que abarcan desde los fundamentos teóricos de la computación hasta el desarrollo de interfaces gráficas de usuario.

La Unidad 1 introduce los cimientos esenciales: historia de la computación, paradigmas de programación, los lenguajes C y C++, programación estructurada y algoritmos, operadores, estructuras de control, funciones y macros. Cada tema incluye ejemplos con código ejecutable y pruebas de escritorio que permiten al estudiante verificar su comprensión paso a paso.

La Unidad 2 profundiza en temas avanzados: análisis de complejidad computacional con notación Big-O, manejo de archivos de texto y binarios, trabajo con múltiples archivos, estructuras `struct` en C, una panorámica de lenguajes modernos (PHP, Java, JavaScript, Go, Rust y Mojo), y los principales frameworks de interfaces gráficas de usuario.

Cada unidad mantiene su identidad visual propia para facilitar la navegación. Las páginas de la Unidad 1 emplean una paleta en tonos **azul académico**, mientras que la Unidad 2 utiliza **verde azulado**, permitiendo distinguirlas de un vistazo.

Se recomienda leer las unidades en orden, ya que los conceptos de C presentados en la Unidad 1 son prerequisite para los temas de archivos y estructuras de la Unidad 2.

Gilma Coaquira Salluca
2026

Contents

Prefacio	3
I Unidad 1 — Fundamentos y Programación en C y C++	
1 Fundamentos e Historia de la Computación	13
1.1 ¿Qué es la Computación?	13
1.2 ¿Por qué surge la Computación?	13
1.3 Historia Breve	13
1.3.1 Acontecimientos Importantes	13
1.4 Estructuras de Datos	14
1.4.1 ¿Por qué son importantes?	14
1.5 Proceso de la Información	14
1.5.1 Entrada	14
1.5.2 Proceso	14
1.5.3 Salida	14
1.6 Del Código Fuente al Hardware	14
2 Paradigmas de Programación	17
2.1 Paradigma Imperativo	17
2.2 Paradigma Procedimental	17
2.3 Paradigma Estructurado	18
2.4 Paradigma Orientado a Objetos	18
2.5 Paradigma Declarativo (Conceptual)	18
2.6 Paradigmas que usa C	18
2.7 Paradigmas que usa C++	19
3 Lenguajes de Programación C y C++	21
3.1 ¿Qué es un Lenguaje de Programación?	21
3.2 ¿Qué es C?	21
3.3 ¿Qué es C++?	21
3.4 Diferencias Claras entre C y C++	21
3.5 Ventajas y Usos	22
3.5.1 Ventajas de C	22
3.5.2 Usos de C	22
3.5.3 Ventajas de C++	22
3.5.4 Usos de C++	22
3.6 Compilación Paso a Paso	22
3.7 Capas de Abstracción en Programación	23
3.8 Tipos de Datos — Tabla Completa	23
3.9 Conversión de Tipos	23

3.9.1	Conversión Implícita (Automática)	23
3.9.2	Conversión Explícita (Casting)	24
4	Programación Estructurada y Algoritmos	25
4.1	¿Qué es un Algoritmo?	25
4.2	Programación Estructurada	25
4.3	Variables	25
4.4	Constantes	25
4.5	Representaciones de Algoritmos	26
4.5.1	Pseudocódigo	26
4.5.2	Diagrama de Flujo	26
4.5.3	Prueba de Escritorio	26
5	Operadores en C y C++	27
5.1	Operadores Aritméticos	27
5.2	Operadores Relacionales	27
5.3	Operadores Lógicos	28
5.4	Operadores de Asignación	28
5.5	Operadores de Incremento y Decremento	28
5.6	Precedencia de Operadores	28
5.6.1	Ejemplo Resuelto con Prueba de Escritorio	29
6	Estructuras de Control	31
6.1	Clasificación	31
6.1.1	Secuenciales	31
6.1.2	Condicionales	31
6.1.3	Repetitivas (Bucles)	32
7	Funciones y Macros	35
7.1	Funciones	35
7.1.1	¿Qué son las funciones?	35
7.1.2	¿Para qué sirven?	35
7.1.3	Partes de una Función	35
7.1.4	Paso por Valor	35
7.1.5	Ejemplo Completo	36
7.2	Macros	36
7.2.1	¿Qué son las macros?	36
7.2.2	Diferencia con Funciones	36
7.2.3	Macros sin Parámetros	36
7.2.4	Macros con Parámetros	37

II Unidad 2 — Complejidad, Archivos, Estructuras y Lenguajes Modernos

8	Análisis de Complejidad Computacional	43
8.1	Notación Big-O	43
8.1.1	Clases de complejidad	43
8.1.2	Reglas de simplificación	44
8.2	Complejidad Temporal y Espacial	44

8.2.1	Complejidad Temporal	44
8.2.2	Complejidad Espacial	45
8.3	Comparativa de Algoritmos	46
8.4	Estructuras de Datos y sus Complejidades	47
9	Manejo de Archivos en C	49
9.1	Escritura Básica de Texto	49
9.2	Lectura Línea por Línea	50
9.3	Archivos Binarios con fwrite y fread	50
10	Manejo de Múltiples Archivos en C	53
10.1	Escribir y Leer Múltiples Archivos	53
10.2	Filtrar, Copiar y Consolidar Archivos	54
10.3	Patrones y Comandos Clave	55
11	Estructuras (struct) en C	57
11.1	Qué es una Estructura	57
11.2	Sintaxis Básica y typedef	57
11.3	Arreglos de Estructuras	58
11.4	Estructuras con Funciones y Archivos — CRUD Completo	59
12	Otros Lenguajes de Programación	61
12.1	PHP	61
12.2	Java	62
12.3	JavaScript	62
12.4	Go	63
12.5	Rust	63
12.6	Mojo	64
12.7	Tabla Comparativa	64
13	Interfaces Gráficas de Usuario (GUI)	65
13.1	Frameworks por Lenguaje	65
13.2	Tkinter — Python	65
13.3	PyQt5 — Python	66
13.4	Electron — JavaScript	66
13.5	Flutter — Dart	66
13.6	Swing y JavaFX — Java	67
13.7	Conceptos Esenciales de Diseño GUI	67
13.7.1	Principios de Usabilidad	67
13.7.2	Hilo Principal y Hilos Secundarios	67
	Referencias	69

Part I

Unidad 1 — Fundamentos y Programación en C y C++

UNIDAD 1

Fundamentos e Historia de la Computación,
Paradigmas, Lenguajes C y C++,
Programación Estructurada y Algoritmos,
Operadores, Estructuras de Control,
Funciones y Macros

Esta unidad cubre los conceptos fundamentales de la computación y los lenguajes C y C++, desde la historia hasta funciones y macros.

Fundamentos e Historia de la Computación

1.1 ¿Qué es la Computación?

La computación es la ciencia que estudia el procesamiento automático de información mediante dispositivos electrónicos y sistemas computacionales. Abarca desde los fundamentos teóricos del procesamiento de datos hasta el diseño de hardware y software para resolver problemas del mundo real.

1.2 ¿Por qué surge la Computación?

La computación surge como respuesta a necesidades fundamentales del ser humano:

- **Cálculos complejos:** La necesidad de realizar operaciones matemáticas extensas y precisas que superaban la capacidad humana.
- **Procesamiento de información:** Manejar grandes volúmenes de datos de manera eficiente.
- **Automatización:** Liberar al ser humano de tareas repetitivas y permitir el enfoque en actividades creativas.
- **Comunicación:** Facilitar el intercambio de información a gran escala.

1.3 Historia Breve

La computación moderna no surgió de la nada; es el resultado de siglos de innovación. Desde los primeros dispositivos mecánicos hasta las computadoras cuánticas, cada etapa representa un salto cualitativo en nuestra capacidad de procesar información.

1.3.1 Acontecimientos Importantes

El Ábaco (2700–2300 a.C.) El primer dispositivo de cálculo conocido, originado en Mesopotamia. Permitía realizar operaciones aritméticas básicas mediante el desplazamiento de cuentas en varillas. Demostró que la computación podía ser asistida por herramientas externas.

Máquina Analítica de Charles Babbage (1837) Considerada el primer diseño de una computadora de propósito general. Aunque nunca se construyó completamente, conceptualizó elementos clave: unidad de control, memoria (almacén), procesador aritmético (molino) y entrada/salida mediante tarjetas perforadas.

Ada Lovelace y los Algoritmos (1843) Augusta Ada King, condesa de Lovelace, escribió el primer algoritmo destinado a ser procesado por una máquina, específicamente para la Máquina Analítica. Es reconocida como la primera programadora de la historia, al comprender que las máquinas podían ir más allá del simple cálculo numérico.

ENIAC (1945) Electronic Numerical Integrator and Computer fue la primera computadora electrónica digital de propósito general. Ocupaba 167 m² y pesaba 27 toneladas. Marcó el inicio de la era de la computación electrónica moderna.

1.4 Estructuras de Datos

Las estructuras de datos son formas de organizar y almacenar información para que pueda ser utilizada de manera eficiente. Niklaus Wirth, creador del lenguaje Pascal, estableció una máxima fundamental:

$$\textit{“Algoritmos + Estructuras de Datos = Programas”}$$

1.4.1 ¿Por qué son importantes?

- Optimizan el uso de memoria
- Mejoran el rendimiento de los algoritmos
- Facilitan el mantenimiento del código
- Permiten modelar problemas del mundo real de manera efectiva

1.5 Proceso de la Información

Todo sistema computacional opera bajo un modelo fundamental de tres etapas:

1.5.1 Entrada

Recepción de datos desde fuentes externas: teclado, archivos, sensores, redes, etc.

1.5.2 Proceso

Transformación de los datos de entrada mediante algoritmos y operaciones lógico-matemáticas para generar resultados útiles.

1.5.3 Salida

Presentación de los resultados procesados al usuario o a otros sistemas: pantalla, impresora, archivos, señales de control, etc.

1.6 Del Código Fuente al Hardware

El proceso mediante el cual un programa escrito por un humano se convierte en instrucciones ejecutables por el hardware sigue estos pasos:

1. **Código Fuente:** El programador escribe instrucciones en un lenguaje de alto nivel (C, C++, Java, Python, etc.) utilizando texto legible por humanos.

2. **Compilador:** Un programa especializado traduce el código fuente a código máquina. En el caso de C/C++, el compilador genera código objeto que luego es enlazado para crear el ejecutable.
3. **Código Máquina:** Secuencia de instrucciones binarias (0s y 1s) que la CPU puede interpretar directamente. Cada arquitectura de procesador tiene su propio conjunto de instrucciones.
4. **Sistema Operativo:** Actúa como intermediario entre el programa y el hardware. Gestiona la memoria, el procesador, los dispositivos de entrada/salida y proporciona servicios al programa.
5. **CPU, RAM y Dispositivos:** La CPU ejecuta las instrucciones, la RAM almacena temporalmente datos y código en ejecución, y los dispositivos periféricos permiten la interacción con el mundo exterior.

2

Paradigmas de Programación

Un paradigma de programación es un enfoque o filosofía para resolver problemas mediante código. Define cómo se estructura el programa, cómo se organiza la lógica y cómo fluye el control.

2.1 Paradigma Imperativo

Concepto: El programador especifica *cómo* realizar una tarea mediante una secuencia de comandos que modifican el estado del programa.

Cómo se programa: Se definen variables que almacenan el estado y se escriben instrucciones que modifican ese estado paso a paso.

```
1 int suma = 0;
2 suma = suma + 5;
3 suma = suma + 3;
4 // suma ahora vale 8
```

Listing 2.1: Ejemplo imperativo

2.2 Paradigma Procedimental

Concepto: Extensión del paradigma imperativo que organiza el código en procedimientos o funciones reutilizables.

Cómo se programa: Se divide el problema en subproblemas, cada uno resuelto por una función independiente.

```
1 void saludar(char *nombre) {
2     printf("Hola, %s\n", nombre);
3 }
4
5 int main() {
6     saludar("Gilma");
7     return 0;
8 }
```

Listing 2.2: Ejemplo procedimental

2.3 Paradigma Estructurado

Concepto: Enfatiza el uso de estructuras de control claras (secuencia, selección, iteración) y evita el uso de `goto`.

```
1 for (int i = 0; i < 5; i++) {
2     if (i % 2 == 0) {
3         printf("%d es par\n", i);
4     } else {
5         printf("%d es impar\n", i);
6     }
7 }
```

Listing 2.3: Ejemplo estructurado

2.4 Paradigma Orientado a Objetos

Concepto: Organiza el código en “objetos” que combinan datos (atributos) y comportamientos (métodos). Se basa en encapsulación, herencia y polimorfismo.

```
1 class Persona {
2     private:
3         string nombre;
4     public:
5         Persona(string n) : nombre(n) {}
6         void saludar() {
7             cout << "Hola, soy " << nombre << endl;
8         }
9 };
10
11 int main() {
12     Persona p("Gilma");
13     p.saludar();
14     return 0;
15 }
```

Listing 2.4: Ejemplo orientado a objetos (C++)

2.5 Paradigma Declarativo (Conceptual)

Concepto: El programador especifica *qué* resultado desea obtener, no *cómo* obtenerlo. El sistema determina la forma de lograrlo.

Ejemplo conceptual (SQL):

```
SELECT nombre FROM estudiantes WHERE edad > 18;
```

2.6 Paradigmas que usa C

C es principalmente un lenguaje:

- **Imperativo:** Permite control directo sobre el estado del programa
- **Procedimental:** Se organiza en funciones

- **Estructurado:** Usa estructuras de control claras

2.7 Paradigmas que usa C++

C++ es un lenguaje multiparadigma que soporta:

- **Imperativo:** Herencia de C
- **Procedimental:** Funciones y procedimientos
- **Estructurado:** Estructuras de control
- **Orientado a Objetos:** Clases, herencia, polimorfismo, encapsulación
- **Genérico:** Mediante templates (plantillas)

3

Lenguajes de Programación C y C++

3.1 ¿Qué es un Lenguaje de Programación?

Un lenguaje de programación es un sistema formal de comunicación entre el programador y la computadora. Proporciona un conjunto de reglas sintácticas y semánticas para expresar algoritmos de manera que puedan ser traducidos a instrucciones ejecutables por el hardware.

3.2 ¿Qué es C?

C es un lenguaje de programación de propósito general desarrollado por Dennis Ritchie entre 1969 y 1973 en los Laboratorios Bell. Fue creado para reescribir el sistema operativo UNIX. Es un lenguaje de nivel medio que combina características de alto nivel (abstracción, estructuras de control) con acceso directo al hardware (punteros, manipulación de memoria).

3.3 ¿Qué es C++?

C++ es un lenguaje de programación desarrollado por Bjarne Stroustrup en 1979 como una extensión de C. Originalmente llamado “C con Clases”, añade programación orientada a objetos, programación genérica mediante templates, y otras características modernas, manteniendo la eficiencia y el control de bajo nivel de C.

3.4 Diferencias Claras entre C y C++

C	C++
Paradigma procedimental	Multiparadigma (POO, genérico, etc.)
No tiene clases ni objetos	Soporte completo para POO
<code>printf/scanf</code> para E/S	<code>iostream (cin/cout)</code>
Gestión manual de memoria	Constructores/destructores automáticos
No tiene referencias	Soporta referencias además de punteros
Sin sobrecarga de funciones	Sobrecarga de funciones y operadores

3.5 Ventajas y Usos

3.5.1 Ventajas de C

- Eficiencia y rendimiento óptimo
- Portabilidad entre plataformas
- Control directo del hardware
- Base para otros lenguajes

3.5.2 Usos de C

- Sistemas operativos (Linux, Windows)
- Drivers y firmware
- Sistemas embebidos
- Programación de bajo nivel

3.5.3 Ventajas de C++

- Abstracción sin pérdida de rendimiento
- Reutilización de código (herencia, templates)
- STL (Standard Template Library)
- Compatibilidad con C

3.5.4 Usos de C++

- Videojuegos (Unreal Engine)
- Aplicaciones de alto rendimiento
- Sistemas en tiempo real
- Software gráfico (Adobe, AutoCAD)

3.6 Compilación Paso a Paso

1. **Preprocesamiento:** El preprocesador expande las directivas (`#include`, `#define`) y genera código fuente expandido.
2. **Compilación:** El compilador traduce el código fuente a lenguaje ensamblador.
3. **Ensamblado:** El ensamblador convierte el código ensamblador en código objeto (binario).
4. **Enlazado:** El enlazador (*linker*) combina los archivos objeto y las bibliotecas para generar el ejecutable final.

3.7 Capas de Abstracción en Programación

1. **Hardware:** Circuitos electrónicos, transistores, puertas lógicas.
2. **Lenguaje Máquina:** Instrucciones binarias específicas del procesador (0s y 1s).
3. **Lenguaje Ensamblador:** Representación simbólica del lenguaje máquina (ADD, MOV, JMP).
4. **Lenguajes de Bajo Nivel (C):** Abstracción mínima, acceso directo a memoria y hardware.
5. **Lenguajes de Medio Nivel (C++):** Balance entre control y abstracción.
6. **Lenguajes de Alto Nivel (Python, Java):** Máxima abstracción, facilidad de uso, menor control directo.
7. **Frameworks y Bibliotecas:** Abstracciones especializadas para dominios específicos.

3.8 Tipos de Datos — Tabla Completa

Tipo	Bytes	Rango	Formato	Signo
char	1	−128 a 127	%c	signed
unsigned char	1	0 a 255	%c	unsigned
short	2	−32 768 a 32 767	%hd	signed
unsigned short	2	0 a 65 535	%hu	unsigned
int	4	−2 147 483 648 a 2 147 483 647	%d	signed
unsigned int	4	0 a 4 294 967 295	%u	unsigned
long	4/8	Varía según arquitectura	%ld	signed
float	4	$\pm 3.4 \times 10^{-38}$ a $\pm 3.4 \times 10^{38}$	%f	signed
double	8	$\pm 1.7 \times 10^{-308}$ a $\pm 1.7 \times 10^{308}$	%lf	signed
long double	10/12/16	Mayor precisión que double	%Lf	signed

3.9 Conversión de Tipos

3.9.1 Conversión Implícita (Automática)

Ocurre cuando se asignan valores de un tipo a una variable de otro tipo compatible, sin intervención del programador.

```

1 int entero = 10;
2 float decimal = entero; // int a float automaticamente
3 // decimal ahora vale 10.0

```

Listing 3.1: Conversión implícita

3.9.2 Conversión Explícita (Casting)

El programador fuerza la conversión mediante un operador de conversión.

```
1 float decimal = 7.8;
2 int entero = (int)decimal; // Casting explícito
3 // entero ahora vale 7 (se pierde la parte decimal)
4
5 double pi = 3.14159;
6 int pi_entero = (int)pi; // pi_entero = 3
7 // Se pierden los decimales .14159
```

Listing 3.2: Casting explícito

Programación Estructurada y Algoritmos

4.1 ¿Qué es un Algoritmo?

Un algoritmo es una secuencia finita y ordenada de pasos lógicos que resuelven un problema específico. Debe cumplir:

- **Finitud:** Debe terminar en un número finito de pasos
- **Precisión:** Cada paso debe estar claramente definido
- **Entrada:** Puede tener cero o más entradas
- **Salida:** Debe producir al menos un resultado
- **Efectividad:** Cada operación debe ser básica y realizable

4.2 Programación Estructurada

Es un paradigma de programación que utiliza únicamente tres estructuras de control: secuencia, selección e iteración. Elimina el uso de `goto` y promueve código claro, legible y mantenible.

4.3 Variables

Una variable es un espacio de memoria identificado por un nombre que almacena un valor que puede cambiar durante la ejecución del programa.

```
1 int edad = 25;  
2 float salario = 3500.50;  
3 char inicial = 'G';
```

Listing 4.1: Declaración de variables

4.4 Constantes

Una constante es un valor que no cambia durante la ejecución del programa.

```
1 const float PI = 3.14159;  
2 const int DIAS_SEMANA = 7;
```

Listing 4.2: Declaración de constantes

4.5 Representaciones de Algoritmos

Para ilustrar las diferentes representaciones usaremos el mismo problema: **calcular el área de un círculo dado su radio.**

4.5.1 Pseudocódigo

```

INICIO
  DECLARAR radio COMO REAL
  DECLARAR area  COMO REAL
  DECLARAR PI = 3.14159 COMO CONSTANTE
  ESCRIBIR "Ingrese el radio del circulo: "
  LEER radio
  area = PI * radio * radio
  ESCRIBIR "El area del circulo es: ", area
FIN

```

4.5.2 Diagrama de Flujo

Un diagrama de flujo utiliza símbolos gráficos estándar:

- **Óvalo:** Inicio / Fin
- **Paralelogramo:** Entrada / Salida
- **Rectángulo:** Proceso
- **Rombo:** Decisión
- **Flechas:** Flujo de control

Para nuestro problema: (1) Inicio $\rightarrow$ (2) Declarar variables $\rightarrow$ (3) Leer radio $\rightarrow$ (4) Calcular $area = \pi \cdot r^2 \rightarrow$ (5) Mostrar área $\rightarrow$ (6) Fin.

4.5.3 Prueba de Escritorio

Es una técnica de verificación manual donde se ejecuta el algoritmo paso a paso con valores específicos.

Datos de entrada: radio = 5.0

Paso	Instrucción	radio	area
1	Declarar variables	–	–
2	PI = 3.14159	–	–
3	Leer radio	5.0	–
4	area = 3.14159 * 5.0 * 5.0	5.0	78.53975
5	Mostrar area	5.0	78.53975

Salida esperada: El área del círculo es: 78.53975

5

Operadores en C y C++

Los operadores son símbolos que indican al compilador realizar operaciones matemáticas, lógicas o de manipulación de datos.

5.1 Operadores Aritméticos

Operador	Descripción	Ejemplo
+	Suma	a + b
-	Resta	a - b
*	Multipliación	a * b
/	División	a / b
%	Módulo (residuo)	a % b

```

1 int a = 10, b = 3;
2 int suma      = a + b; // suma      = 13
3 int resta     = a - b; // resta     = 7
4 int producto  = a * b; // producto = 30
5 int division  = a / b; // division = 3 (division entera)
6 int residuo   = a % b; // residuo   = 1

```

Listing 5.1: Operadores aritméticos con resultados

5.2 Operadores Relacionales

Comparan dos valores y devuelven verdadero (1) o falso (0).

Operador	Descripción	Ejemplo
==	Igual a	a == b
!=	Diferente de	a != b
>	Mayor que	a > b
<	Menor que	a < b
>=	Mayor o igual que	a >= b
<=	Menor o igual que	a <= b

5.3 Operadores Lógicos

Operador	Descripción	Ejemplo
&&	AND lógico	a && b
	OR lógico	a b
!	NOT lógico	!a

5.4 Operadores de Asignación

Operador	Descripción	Equivalente
=	Asignación simple	a = 5
+=	Suma y asigna	a = a + 3
-=	Resta y asigna	a = a - 3
*=	Multiplica y asigna	a = a * 3
/=	Divide y asigna	a = a / 3

5.5 Operadores de Incremento y Decremento

Operador	Descripción	Efecto
++	Incremento	a++ o ++a (a = a + 1)
--	Decremento	a- o -a (a = a - 1)

```

1 int a = 5;
2 int b = a++; // b = 5, a = 6 (post-incremento)
3 int c = ++a; // c = 7, a = 7 (pre-incremento)

```

Listing 5.2: Incremento y decremento

5.6 Precedencia de Operadores

La precedencia determina el orden en que se evalúan las operaciones.

Nivel	Operadores	Asociatividad
1	() [] -> .	Izq. a Der.
2	! ++ -	Der. a Izq.
3	* / %	Izq. a Der.
4	+ -	Izq. a Der.
5	< <= > >=	Izq. a Der.
6	== !=	Izq. a Der.
7	&&	Izq. a Der.
8		Izq. a Der.
9	= += -= *= /=	Der. a Izq.

5.6.1 Ejemplo Resuelto con Prueba de Escritorio

Expresión: $\text{resultado} = 5 + 3 * 2 - 8 / 4$

Paso	Operación	Resultado Parcial
1	$3 * 2$	6
2	$8 / 4$	2
3	$5 + 6$	11
4	$11 - 2$	9

```
1 #include <stdio.h>
2
3 int main() {
4     int resultado = 5 + 3 * 2 - 8 / 4;
5     printf("Resultado: %d\n", resultado);
6     return 0;
7 }
```

Listing 5.3: Programa completo — resultado = 9

6

Estructuras de Control

Las estructuras de control determinan el flujo de ejecución de un programa.

6.1 Clasificación

6.1.1 Secuenciales

Las instrucciones se ejecutan en orden, de arriba hacia abajo, en el orden en que aparecen en el código.

```
1  #include <stdio.h>
2
3  int main() {
4      int a = 5;
5      int b = 10;
6      int suma = a + b;
7      printf("La suma es: %d\n", suma);
8      return 0;
9  }
10 // Salida: La suma es: 15
```

Listing 6.1: Estructura secuencial

6.1.2 Condicionales

if-else

Evalúa una condición; si es verdadera ejecuta un bloque, si es falsa ejecuta otro.

```
1  #include <stdio.h>
2
3  int main() {
4      int edad = 20;
5
6      if (edad >= 18) {
7          printf("Eres mayor de edad\n");
8      } else {
9          printf("Eres menor de edad\n");
10     }
11
12     return 0;
13 }
```

```
14 // Salida: Eres mayor de edad
```

Listing 6.2: Estructura if-else

switch

Evalúa una expresión y ejecuta el caso que coincida con su valor.

```
1 #include <stdio.h>
2
3 int main() {
4     int dia = 3;
5
6     switch (dia) {
7         case 1: printf("Lunes\n");    break;
8         case 2: printf("Martes\n");   break;
9         case 3: printf("Miercoles\n");break;
10        default: printf("Dia invalido\n");
11    }
12
13    return 0;
14 }
15 // Salida: Miercoles
```

Listing 6.3: Estructura switch

6.1.3 Repetitivas (Bucles)

while

Repite un bloque mientras la condición sea verdadera. La condición se evalúa *antes* de cada iteración.

```
1 #include <stdio.h>
2
3 int main() {
4     int i = 1;
5
6     while (i <= 5) {
7         printf("%d ", i);
8         i++;
9     }
10    printf("\n");
11
12    return 0;
13 }
14 // Salida: 1 2 3 4 5
```

Listing 6.4: Bucle while

for

Bucle con inicialización, condición e incremento en una sola línea. Ideal cuando se conoce el número de iteraciones.


```
1  #include <stdio.h>
2
3  int main() {
4      for (int i = 1; i <= 5; i++) {
5          printf("%d ", i);
6      }
7      printf("\n");
8
9      return 0;
10 }
11 // Salida: 1 2 3 4 5
```

Listing 6.5: Bucle for

do-while

Similar a **while**, pero la condición se evalúa *después* de cada iteración, garantizando al menos una ejecución.

```
1  #include <stdio.h>
2
3  int main() {
4      int i = 1;
5
6      do {
7          printf("%d ", i);
8          i++;
9      } while (i <= 5);
10
11     printf("\n");
12     return 0;
13 }
14 // Salida: 1 2 3 4 5
```

Listing 6.6: Bucle do-while

7

Funciones y Macros

7.1 Funciones

7.1.1 ¿Qué son las funciones?

Una función es un bloque de código independiente que realiza una tarea específica. Puede recibir parámetros (entradas) y devolver un resultado (salida).

7.1.2 ¿Para qué sirven?

- Reutilización de código
- Organización lógica del programa
- Facilitan el mantenimiento
- Abstracción de complejidad
- Trabajo en equipo (diferentes funciones por diferentes programadores)

7.1.3 Partes de una Función

```
1 tipo_retorno nombre_funcion(tipo_param1 param1, tipo_param2
   param2) {
2     // Cuerpo de la funcion
3     return valor;
4 }
```

Listing 7.1: Estructura general de una función

- **tipo_retorno**: Tipo de dato que devuelve (`int`, `float`, `void`, etc.)
- **nombre_funcion**: Identificador de la función
- **parámetros**: Valores de entrada (opcionales)
- **return**: Valor que devuelve la función

7.1.4 Paso por Valor

En C, los parámetros se pasan por valor, es decir, se copia el valor de la variable. Los cambios dentro de la función **no afectan** a la variable original.

7.1.5 Ejemplo Completo

```
1 #include <stdio.h>
2
3 // Declaracion de la funcion
4 int calcularCuadrado(int numero);
5
6 int main() {
7     int num = 5;
8     int resultado;
9
10    resultado = calcularCuadrado(num);
11    printf("El cuadrado de %d es %d\n", num, resultado);
12
13    return 0;
14 }
15
16 // Definicion de la funcion
17 int calcularCuadrado(int numero) {
18     return numero * numero;
19 }
20 // Salida: El cuadrado de 5 es 25
```

Listing 7.2: Función completa — calcular cuadrado

7.2 Macros

7.2.1 ¿Qué son las macros?

Las macros son directivas del preprocesador que realizan sustituciones de texto antes de la compilación. Se definen con `#define`.

7.2.2 Diferencia con Funciones

Funciones	Macros
Se ejecutan en tiempo de ejecución	Se expanden en tiempo de compilación
Tienen verificación de tipos	No verifican tipos
Ocupan espacio en memoria	No ocupan espacio (sustitución)
Más lentas (llamada a función)	Más rápidas (código inline)
Pueden ser depuradas	Difíciles de depurar

7.2.3 Macros sin Parámetros

```
1 #include <stdio.h>
2
3 #define PI      3.14159
4 #define SALUDO  "Hola, mundo"
5 #define MAX     100
```

```
6
7 int main() {
8     printf("El valor de PI es: %f\n", PI);
9     printf("%s\n", SALUDO);
10    printf("Maximo: %d\n", MAX);
11
12    return 0;
13 }
```

Listing 7.3: Macros sin parámetros

7.2.4 Macros con Parámetros

```
1 #include <stdio.h>
2
3 #define CUADRADO(x)      ((x) * (x))
4 #define MAXIMO(a,b)      ((a) > (b) ? (a) : (b))
5 #define AREA_CIRCULO(r)  (3.14159 * (r) * (r))
6
7 int main() {
8     int num = 5;
9     int a = 10, b = 20;
10    float radio = 3.0;
11
12    printf("Cuadrado de %d: %d\n", num, CUADRADO(num));
13    printf("Maximo entre %d y %d: %d\n", a, b, MAXIMO(a, b));
14    printf("Area del circulo radio %.1f: %.2f\n",
15           radio, AREA_CIRCULO(radio));
16
17    return 0;
18 }
19 // Cuadrado de 5: 25
20 // Maximo entre 10 y 20: 20
21 // Area del circulo radio 3.0: 28.27
```

Listing 7.4: Macros con parámetros

Part II

Unidad 2 — Complejidad, Archivos, Estructuras y Lenguajes Modernos

UNIDAD 2

Análisis de Complejidad Computacional,
Manejo de Archivos en C,
Manejo de Múltiples Archivos,
Estructuras **struct** en C,
Otros Lenguajes de Programación,
Interfaces Gráficas de Usuario (GUI)

Esta unidad profundiza en temas avanzados de C y presenta una panorámica de lenguajes modernos y frameworks de interfaces gráficas.

Análisis de Complejidad Computacional

El análisis de complejidad mide los recursos que consume un algoritmo —tiempo de ejecución y memoria— en función del tamaño de la entrada n . Permite comparar soluciones, predecir su comportamiento ante entradas grandes y tomar decisiones de diseño fundamentadas antes de implementar.

8.1 Notación Big-O

La notación Big-O expresa la cota superior asintótica de una función: describe el crecimiento del algoritmo en el peor caso, ignorando constantes multiplicativas y términos de menor orden.

Definición formal

$f(n) = O(g(n))$ si existen constantes $c > 0$ y $n_0 \geq 0$ tales que $f(n) \leq c \cdot g(n)$ para todo $n \geq n_0$.

Además de Big-O existen otras notaciones complementarias:

- **Notación Ω (Omega):** cota inferior. Describe el mejor caso.
- **Notación Θ (Theta):** cota ajustada. El algoritmo crece exactamente como $g(n)$.

8.1.1 Clases de complejidad

Clase	Nombre	Ejemplo clásico	n=1 000
$O(1)$	Constante	Acceso a arreglo, tabla hash	1 op.
$O(\log n)$	Logarítmica	Búsqueda binaria	≈ 10
$O(n)$	Lineal	Recorrido de lista	1 000
$O(n \log n)$	Lineal-log	Merge sort, Heap sort	$\approx 10\,000$
$O(n^2)$	Cuadrática	Burbuja, Selección	10^6
$O(n^3)$	Cúbica	Multiplicación naïve	10^9
$O(2^n)$	Exponencial	Subconjuntos, backtracking	$\approx 10^{301}$
$O(n!)$	Factorial	Permutaciones	$\gg 10^{2500}$

Nota importante

A partir de $O(n^2)$ el rendimiento se degrada drásticamente. Con $n = 100\,000$, un algoritmo $O(n^2)$ realiza 10^{10} operaciones, mientras que uno $O(n \log n)$ realiza solo $\approx 1,7 \times 10^6$.

8.1.2 Reglas de simplificación

1. **Eliminar constantes:** $O(5n) = O(n)$, $O(1000) = O(1)$.
2. **Eliminar términos de menor orden:** $O(n^2 + n) = O(n^2)$.
3. **Suma de bloques secuenciales:** $O(f) + O(g) = O(\max(f, g))$.
4. **Producto de bloques anidados:** $O(f) \cdot O(g) = O(f \cdot g)$.

8.2 Complejidad Temporal y Espacial**8.2.1 Complejidad Temporal**

Mide el número de operaciones elementales en función de n .

```

1  /* Mejor caso: O(1) | Peor/Promedio: O(n) */
2  int busqueda_lineal(int arr[], int n, int obj) {
3      for (int i = 0; i < n; i++) {
4          if (arr[i] == obj)
5              return i;
6      }
7      return -1;
8  }
```

Listing 8.1: Búsqueda lineal — $O(n)$ tiempo, $O(1)$ espacio

```

1  /* El arreglo DEBE estar ordenado. */
2  int busqueda_binaria(int arr[], int n, int obj) {
3      int izq = 0, der = n - 1;
4
5      while (izq <= der) {
6          int medio = izq + (der - izq) / 2;
7
8          if (arr[medio] == obj)
9              return medio;
10         else if (arr[medio] < obj)
11             izq = medio + 1;
12         else
13             der = medio - 1;
14     }
15     return -1;
16 }
```

Listing 8.2: Búsqueda binaria — $O(\log n)$ tiempo, $O(1)$ espacio

```

1 void burbuja(int arr[], int n) {
2     for (int i = 0; i < n - 1; i++) {
3         int intercambio = 0;
4         for (int j = 0; j < n - 1 - i; j++) {
5             if (arr[j] > arr[j + 1]) {
6                 int tmp = arr[j];
7                 arr[j] = arr[j + 1];
8                 arr[j + 1] = tmp;
9                 intercambio = 1;
10            }
11        }
12        if (!intercambio) break;
13    }
14 }

```

Listing 8.3: Ordenamiento burbuja — $O(n^2)$ tiempo

```

1 #include <stdlib.h>
2
3 void merge(int arr[], int izq, int medio, int der) {
4     int n1 = medio - izq + 1;
5     int n2 = der - medio;
6     int *L = (int *)malloc(n1 * sizeof(int));
7     int *R = (int *)malloc(n2 * sizeof(int));
8
9     for (int i = 0; i < n1; i++) L[i] = arr[izq + i];
10    for (int j = 0; j < n2; j++) R[j] = arr[medio + 1 + j];
11
12    int i = 0, j = 0, k = izq;
13    while (i < n1 && j < n2)
14        arr[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];
15    while (i < n1) arr[k++] = L[i++];
16    while (j < n2) arr[k++] = R[j++];
17
18    free(L); free(R);
19 }
20
21 void merge_sort(int arr[], int izq, int der) {
22     if (izq < der) {
23         int medio = izq + (der - izq) / 2;
24         merge_sort(arr, izq, medio);
25         merge_sort(arr, medio + 1, der);
26         merge(arr, izq, medio, der);
27     }
28 }
29 /*  $T(n) = 2T(n/2) + O(n) \Rightarrow T(n) = O(n \log n)$  */

```

Listing 8.4: Merge sort — $O(n \log n)$ tiempo, $O(n)$ espacio

8.2.2 Complejidad Espacial

Mide la memoria adicional utilizada por el algoritmo.

```

1  /* RECURSIVO:  $O(2^n)$  tiempo,  $O(n)$  espacio (pila de llamadas) */
2  long long fib_recursivo(int n) {
3      if (n <= 1) return n;
4      return fib_recursivo(n - 1) + fib_recursivo(n - 2);
5  }
6
7  /* ITERATIVO (optimo):  $O(n)$  tiempo,  $O(1)$  espacio */
8  long long fib_iterativo(int n) {
9      if (n <= 1) return n;
10     long long a = 0, b = 1;
11     for (int i = 2; i <= n; i++) {
12         long long c = a + b;
13         a = b;
14         b = c;
15     }
16     return b;
17 }
18
19 /* MEMOIZACION:  $O(n)$  tiempo,  $O(n)$  espacio */
20 long long fib_memo(int n, long long memo[]) {
21     if (n <= 1) return n;
22     if (memo[n] != -1) return memo[n];
23     memo[n] = fib_memo(n-1, memo) + fib_memo(n-2, memo);
24     return memo[n];
25 }

```

Listing 8.5: Fibonacci: recursivo vs iterativo

8.3 Comparativa de Algoritmos

Algoritmo	Mejor	Promedio	Peor	Espacio	Estable
Burbuja	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Sí
Selección	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Inserción	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Sí
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Sí
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No
Búsqueda lineal	$O(1)$	$O(n)$	$O(n)$	$O(1)$	—
Búsqueda binaria	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$	—

8.4 Estructuras de Datos y sus Complejidades

Estructura	Acceso	Búsqueda	Inserción	Eliminación
Arreglo dinámico	$O(1)$	$O(n)$	$O(1)^*$	$O(n)$
Lista enlazada	$O(n)$	$O(n)$	$O(1)$	$O(1)^{**}$
Pila / Cola	$O(n)$	$O(n)$	$O(1)$	$O(1)$
Tabla hash	$O(1)^*$	$O(1)^*$	$O(1)^*$	$O(1)^*$
Árbol BST	$O(\log n)^*$	$O(\log n)^*$	$O(\log n)^*$	$O(\log n)^*$
Árbol AVL	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$
Heap (mín/máx)	$O(1)^{***}$	$O(n)$	$O(\log n)$	$O(\log n)$

* Promedio ** Con puntero al nodo *** Solo mínimo o máximo

9

Manejo de Archivos en C

En C el manejo de archivos se realiza mediante punteros de tipo `FILE*` definidos en `<stdio.h>`. El programador controla explícitamente la apertura, todas las operaciones y el cierre.

Nota importante

Si el archivo no se cierra, los datos pueden quedar retenidos en el buffer interno y perderse. Nunca operar sobre un `FILE* NULL`. Siempre verificar el valor de retorno de `fopen`.

Modo	Descripción	Crea archivo
"r"	Lectura de texto (debe existir)	No
"w"	Escritura de texto (trunca o crea)	Sí
"a"	Append: agrega al final	Sí
"r+"	Lectura y escritura (sin truncar)	No
"rb"	Lectura binaria	No
"wb"	Escritura binaria (trunca o crea)	Sí
"ab"	Append binario	Sí

9.1 Escritura Básica de Texto

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main(void) {
5      FILE *fp = fopen("notas.txt", "w");
6      if (fp == NULL) {
7          perror("fopen");
8          return EXIT_FAILURE;
9      }
10
11     fprintf(fp, "Lenguajes de Programacion I\n");
12     fprintf(fp, "Autora: Gilma Coaquira Salluca\n");
13     fprintf(fp, "Año: %d\n", 2026);
14     fputs("Capítulo 1: Complejidad Computacional\n", fp);

```

```
15     fputs("Capítulo 2: Manejo de Archivos en C\n", fp);
16
17     fclose(fp); /* OBLIGATORIO: vacia el buffer y cierra */
18     printf("Archivo notas.txt creado correctamente.\n");
19
20     /* Modo "a" (append): agrega sin borrar */
21     fp = fopen("notas.txt", "a");
22     if (!fp) { perror("fopen"); return EXIT_FAILURE; }
23     fprintf(fp, "Capítulo 3: Estructuras en C\n");
24     fclose(fp);
25
26     return EXIT_SUCCESS;
27 }
```

Listing 9.1: Escritura con fprintf y fputs

9.2 Lectura Línea por Línea

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  int main(void) {
6      FILE *fp = fopen("notas.txt", "r");
7      if (!fp) { perror("fopen"); return EXIT_FAILURE; }
8
9      char linea[256];
10     int numero = 1;
11
12     printf("=== Contenido de notas.txt ===\n");
13
14     while (fgets(linea, sizeof(linea), fp) != NULL) {
15         size_t len = strlen(linea);
16         if (len > 0 && linea[len - 1] == '\n')
17             linea[len - 1] = '\0';
18         printf("[%02d] %s\n", numero++, linea);
19     }
20
21     fclose(fp);
22     return EXIT_SUCCESS;
23 }
```

Listing 9.2: Lectura con fgets y fscanf

9.3 Archivos Binarios con fwrite y fread

Los archivos binarios almacenan los bytes exactos de las variables en memoria. Son más compactos y rápidos que los de texto, pero no son legibles directamente por humanos.

```
1  #include <stdio.h>
2  #include <stdlib.h>
```

```

3  #include <string.h>
4
5  typedef struct {
6      int    id;
7      char   nombre[40];
8      float  nota;
9  } Estudiante;
10
11 void guardar_estudiantes(const char *ruta,
12                          Estudiante arr[], int n) {
13     FILE *fp = fopen(ruta, "wb");
14     if (!fp) { perror("fopen"); exit(EXIT_FAILURE); }
15
16     fwrite(&n, sizeof(int), 1, fp);
17     size_t escritos = fwrite(arr, sizeof(Estudiante), n, fp);
18     printf("Escritos: %zu registros en '%s'\n", escritos, ruta);
19     fclose(fp);
20 }
21
22 int leer_estudiantes(const char *ruta,
23                      Estudiante arr[], int max) {
24     FILE *fp = fopen(ruta, "rb");
25     if (!fp) { perror("fopen"); return -1; }
26
27     int n = 0;
28     fread(&n, sizeof(int), 1, fp);
29     if (n > max) n = max;
30
31     size_t leidos = fread(arr, sizeof(Estudiante), n, fp);
32     fclose(fp);
33     return (int)leidos;
34 }
35
36 int main(void) {
37     Estudiante alumnos[] = {
38         {1, "Ana Garcia", 9.5f},
39         {2, "Luis Perez", 8.2f},
40         {3, "Maria Torres", 7.8f},
41     };
42     int n = sizeof(alumnos) / sizeof(alumnos[0]);
43
44     guardar_estudiantes("alumnos.bin", alumnos, n);
45
46     Estudiante leidos[10];
47     int total = leer_estudiantes("alumnos.bin", leidos, 10);
48
49     for (int i = 0; i < total; i++)
50         printf("%-5d %-22s %.1f\n",
51             leidos[i].id, leidos[i].nombre, leidos[i].nota);
52
53     return EXIT_SUCCESS;

```

```
54 }
```

Listing 9.3: Escritura y lectura binaria de structs

10

Manejo de Múltiples Archivos en C

En aplicaciones reales es común trabajar con varios archivos: generarlos programáticamente, filtrar su contenido, copiarlos o consolidar varios en uno solo.

10.1 Escribir y Leer Múltiples Archivos

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  #define NUM_ARCHIVOS 4
6
7  void crear_archivos(void) {
8      const char *materias[] = {
9          "Calculo", "Algebra", "Programacion", "Fisica"
10     };
11     int notas[][3] = {
12         {85, 90, 78}, {92, 88, 95}, {98, 97, 99}, {75, 82, 80}
13     };
14
15     for (int i = 0; i < NUM_ARCHIVOS; i++) {
16         char nombre[64];
17         snprintf(nombre, sizeof(nombre), "materia_%d.txt", i +
18             1);
19
20         FILE *fp = fopen(nombre, "w");
21         if (!fp) { perror("fopen"); continue; }
22
23         fprintf(fp, "Materia: %s\n", materias[i]);
24         fprintf(fp, "Notas: ");
25         int suma = 0;
26         for (int j = 0; j < 3; j++) {
27             fprintf(fp, "%d ", notas[i][j]);
28             suma += notas[i][j];
29         }
30         fprintf(fp, "\nPromedio: %.2f\n", suma / 3.0);
31         fclose(fp);
32         printf("Creado: %s\n", nombre);
33     }
```

```
33 }
```

Listing 10.1: Crear y leer varios archivos con nombres dinámicos

10.2 Filtrar, Copiar y Consolidar Archivos

```
1  /* Copia origen a destino byte a byte */
2  int copiar_archivo(const char *origen, const char *destino) {
3      FILE *src = fopen(origen, "rb");
4      FILE *dst = fopen(destino, "wb");
5      if (!src || !dst) { perror("archivo"); return -1; }
6
7      char buf[4096]; size_t n;
8      while ((n = fread(buf, 1, sizeof(buf), src)) > 0)
9          fwrite(buf, 1, n, dst);
10
11     fclose(src); fclose(dst);
12     return 0;
13 }
14
15 /* Filtra líneas que contienen 'patron' */
16 int filtrar_lineas(const char *entrada, const char *salida,
17                   const char *patron) {
18     FILE *fin = fopen(entrada, "r");
19     FILE *fout = fopen(salida, "w");
20     if (!fin || !fout) { perror("filtrar"); return -1; }
21
22     char linea[512];
23     int encontradas = 0;
24
25     while (fgets(linea, sizeof(linea), fin)) {
26         if (strstr(linea, patron)) {
27             fputs(linea, fout);
28             encontradas++;
29         }
30     }
31     fclose(fin); fclose(fout);
32     return encontradas;
33 }
```

Listing 10.2: Copiar, filtrar y consolidar

10.3 Patrones y Comandos Clave

Patrones fundamentales de manejo de archivos en C

1. **Abrir–verificar–operar–cerrar:** Siempre verificar que `fopen` no retorne `NULL`.
2. **Lectura segura:** Preferir `fgets` sobre `gets`.
3. **Flush explícito:** Llamar a `fflush(fp)` antes de leer un archivo recién escrito.
4. **Acceso aleatorio:** `fseek()` + `ftell()` para posicionamiento.
5. **Archivos temporales:** Usar `tmpfile()` para archivos intermedios.

11

Estructuras (struct) en C

Una estructura (**struct**) permite agrupar variables de distintos tipos bajo un mismo nombre, formando un tipo de dato compuesto. Son el mecanismo principal de C para modelar entidades del mundo real.

11.1 Qué es una Estructura

Definición

Una **struct** en C es una colección de campos de tipos posiblemente distintos, almacenados de forma contigua en memoria. Cada campo se accede con el operador punto (.) o flecha (->) si se trabaja con punteros.

11.2 Sintaxis Básica y typedef

```
1  #include <stdio.h>
2  #include <string.h>
3
4  /* Sin typedef */
5  struct Punto {
6      double x;
7      double y;
8  };
9
10 /* Con typedef: evita escribir "struct" cada vez */
11 typedef struct {
12     int id;
13     char nombre[50];
14     float nota;
15     int activo;
16 } Estudiante;
17
18 int main(void) {
19     struct Punto p1 = { .x = 3.0, .y = 4.0 };
20     printf("p1 = (%.1f, %.1f)\n", p1.x, p1.y);
21
22     Estudiante e1 = { .id = 1, .nombre = "Ana Garcia",
23                      .nota = 9.5f, .activo = 1 };
24 }
```

```
25     Estudiante e2;
26     e2.id = 2; e2.nota = 8.0f; e2.activo = 1;
27     strncpy(e2.nombre, "Luis Perez", sizeof(e2.nombre) - 1);
28     e2.nombre[sizeof(e2.nombre) - 1] = '\0';
29
30     /* Puntero a estructura: operador -> */
31     Estudiante *ptr = &e1;
32     printf("Via puntero: %s = %.1f\n", ptr->nombre, ptr->nota);
33
34     return 0;
35 }
```

Listing 11.1: Declaración, inicialización y acceso a estructuras

11.3 Arreglos de Estructuras

```
1  #include <stdio.h>
2  #include <string.h>
3
4  #define MAX 100
5
6  typedef struct {
7      int    id;
8      char   nombre[50];
9      float  nota;
10 } Estudiante;
11
12 /* Ordenar por nota descendente */
13 void ordenar_por_nota(Estudiante arr[], int n) {
14     for (int i = 0; i < n - 1; i++)
15         for (int j = 0; j < n - 1 - i; j++)
16             if (arr[j].nota < arr[j + 1].nota) {
17                 Estudiante tmp = arr[j];
18                 arr[j] = arr[j + 1];
19                 arr[j + 1] = tmp;
20             }
21 }
22
23 float promedio(Estudiante arr[], int n) {
24     float suma = 0;
25     for (int i = 0; i < n; i++) suma += arr[i].nota;
26     return suma / n;
27 }
28
29 int main(void) {
30     Estudiante clase[MAX];
31     int n = 5;
32
33     clase[0] = (Estudiante){1, "Ana Garcia", 9.5f};
34     clase[1] = (Estudiante){2, "Luis Perez", 8.2f};
35     clase[2] = (Estudiante){3, "Maria Torres", 7.8f};
```

```

36     clase[3] = (Estudiante){4, "Carlos Rios", 6.5f};
37     clase[4] = (Estudiante){5, "Sofia Luna", 9.0f};
38
39     ordenar_por_nota(clase, n);
40     printf("Promedio del grupo: %.2f\n", promedio(clase, n));
41
42     return 0;
43 }

```

Listing 11.2: Arreglo de estructuras: búsqueda y ordenamiento

11.4 Estructuras con Funciones y Archivos — CRUD Completo

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  #define RUTA_DB  "base_datos.bin"
6  #define MAX_RECS 200
7
8  typedef struct {
9      int    id;
10     char   nombre[50];
11     char   carrera[40];
12     float  nota;
13     int    activo;    /* 0 = eliminado logicamente */
14 } Registro;
15
16 int cargar(Registro arr[], int max) {
17     FILE *fp = fopen(RUTA_DB, "rb");
18     if (!fp) return 0;
19     int n = 0;
20     Registro r;
21     while (fread(&r, sizeof(Registro), 1, fp) == 1)
22         if (r.activo && n < max) arr[n++] = r;
23     fclose(fp);
24     return n;
25 }
26
27 void guardar(Registro arr[], int n) {
28     FILE *fp = fopen(RUTA_DB, "wb");
29     if (!fp) { perror("guardar"); return; }
30     fwrite(arr, sizeof(Registro), n, fp);
31     fclose(fp);
32 }
33
34 void agregar(Registro arr[], int *n,
35             int id, const char *nombre,
36             const char *carrera, float nota) {

```

```
37     Registro r = {0};
38     r.id = id; r.nota = nota; r.activo = 1;
39     strncpy(r.nombre, nombre, sizeof(r.nombre) - 1);
40     strncpy(r.carrera, carrera, sizeof(r.carrera) - 1);
41     arr[(*n)++] = r;
42     guardar(arr, *n);
43 }
44
45 int eliminar(Registro arr[], int n, int id) {
46     for (int i = 0; i < n; i++)
47         if (arr[i].id == id && arr[i].activo) {
48             arr[i].activo = 0;
49             guardar(arr, n);
50             return 1;
51         }
52     return 0;
53 }
```

Listing 11.3: CRUD completo con structs y archivos binarios

12

Otros Lenguajes de Programación

Los lenguajes de programación modernos han evolucionado para atender necesidades específicas: rendimiento extremo, seguridad de memoria, concurrencia eficiente o productividad del programador.

12.1 PHP

PHP (Hypertext Preprocessor) es el lenguaje del lado del servidor más extendido en la web, impulsando cerca del 43% de los sitios (WordPress, Laravel, Symfony). PHP 8 introdujo JIT, tipos más estrictos, expresión `match` y enumeraciones.

```
1  <?php
2  declare(strict_types=1);
3
4  class Producto {
5      public function __construct(
6          public readonly int    $id,
7          public readonly string $nombre,
8          private float $precio,
9          private int    $stock = 0
10     ) {}
11
12     public function getPrecio(): float { return $this->precio; }
13
14     public function estado(): string {
15         return match(true) {
16             $this->stock === 0 => 'Agotado',
17             $this->stock < 5  => 'Stock critico',
18             $this->stock < 20 => 'Stock bajo',
19             default          => 'Disponible',
20         };
21     }
22 }
23
24 enum Categoria: string {
25     case ELECTRONICA = 'electronica';
26     case ROPA         = 'ropa';
27     case ALIMENTOS    = 'alimentos';
28 }
```

Listing 12.1: PHP 8 — clases, match y enumeraciones

12.2 Java

Java (1995, Sun Microsystems) introdujo la portabilidad mediante la JVM. Es el lenguaje dominante en aplicaciones empresariales (Spring Boot), Android y sistemas financieros. Java 17+ trae records, sealed classes y pattern matching.

```
1 import java.util.*;
2 import java.util.stream.*;
3
4 abstract class Figura {
5     protected final String color;
6     protected Figura(String color) { this.color = color; }
7     public abstract double area();
8 }
9
10 class Circulo extends Figura {
11     private final double radio;
12     public Circulo(double r, String c) { super(c); radio = r; }
13     public double area() { return Math.PI * radio * radio; }
14 }
15
16 /* Record (Java 16+): clase inmutable compacta */
17 record Punto(double x, double y) {
18     double distancia(Punto otro) {
19         return Math.hypot(x - otro.x(), y - otro.y());
20     }
21 }
22
23 // Stream API: filtrar, ordenar, recopilar
24 List<Figura> grandes = figuras.stream()
25     .filter(f -> f.area() > 20)
26     .sorted(Comparator.comparingDouble(Figura::area).reversed())
27     .collect(Collectors.toList());
```

Listing 12.2: Java — OOP, streams y concurrencia

12.3 JavaScript

JavaScript (1995, Netscape) es el único lenguaje nativo de los navegadores. Node.js llevó JS al servidor. Las características modernas más importantes son: clases, async/await, módulos ES, campos privados y generadores.

```
1 /* Generador infinito de Fibonacci */
2 function* fibonacci() {
3     let [a, b] = [0, 1];
4     while (true) { yield a; [a, b] = [b, a + b]; }
5 }
6
```

```

7  /* Async / Await */
8  async function cargarDatos(urls) {
9      const resultados = await Promise.allSettled(
10         urls.map(url => fetch(url).then(r => r.json()))
11     );
12     return resultados.map((r, i) => ({
13         url:    urls[i],
14         exito:  r.status === 'fulfilled',
15         datos:  r.status === 'fulfilled' ? r.value : null,
16     }));
17 }

```

Listing 12.3: JavaScript ES2023 — clases, async/await y generadores

12.4 Go

Go (2009, Google) fue diseñado por Rob Pike, Robert Griesemer y Ken Thompson. Prioriza simplicidad, compilación rápida y concurrencia eficiente mediante goroutines y channels. Docker, Kubernetes y Terraform están escritos en Go.

```

1  /* Interfaz: Go no usa herencia, usa composicion */
2  type Geometria interface {
3      Area()      float64
4      Perimetro() float64
5  }
6
7  /* Worker pool con goroutines */
8  func worker(id int, jobs <-chan Trabajo,
9      res chan<- Resultado, wg *sync.WaitGroup) {
10     defer wg.Done()
11     for t := range jobs {
12         res <- Resultado{ID: t.ID, Salida: math.Sqrt(t.Valor)}
13     }
14 }

```

Listing 12.4: Go — interfaces y goroutines (sintaxis representativa)

12.5 Rust

Rust (2010, Mozilla) resuelve el problema de seguridad de memoria de C/C++ sin usar garbage collector. Su sistema de *ownership* y *borrowing* garantiza en tiempo de compilación que no existan punteros colgantes ni *data races*. Es el lenguaje más querido por desarrolladores (Stack Overflow Survey) desde 2016.

```

1  /* Enum con datos (ADT algebraico) */
2  enum Forma {
3      Circulo      { radio: f64 },
4      Rectangulo { ancho: f64, alto: f64 },
5  }
6
7  impl Forma {
8      fn area(&self) -> f64 {

```

```

9      match self {
10          Forma::Circulo    { radio }      => PI * radio *
              radio,
11          Forma::Rectangulo { ancho, alto } => ancho * alto,
12      }
13  }
14 }
15
16 /* Manejo de errores con Result */
17 fn crear(radio: f64) -> Result<Forma, Error> {
18     if radio < 0.0 { return Err(Error::DimensionNegativa(radio))
19         ; }
20     Ok(Forma::Circulo { radio })
21 }

```

Listing 12.5: Rust — ownership, traits y manejo de errores (sintaxis representativa)

12.6 Mojo

Mojo (2023, Modular Inc.) fue creado por Chris Lattner. Es un superset de Python que compila a código nativo mediante LLVM, logrando rendimientos comparables a C con la sintaxis de Python. Es el lenguaje emergente en el ecosistema de Inteligencia Artificial.

12.7 Tabla Comparativa

Criterio	PHP	Java	JS	Go	Rust	Mojo
Año	1994	1995	1995	2009	2010	2023
Paradigma	Multi	OOP	Multi	Imp.	Multi	Multi
Tipado	Din.	Est.	Din.	Est.	Est.	Est.
Gestión mem.	GC	GC	GC	GC	Ownership	Sin GC
Rendimiento	Bajo	Alto	Medio	Alto	Máx.	Máx.
Uso principal	Web	Emp.	Web/Full	DevOps	Sistemas	IA/HPC

Interfaces Gráficas de Usuario (GUI)

Una interfaz gráfica de usuario (GUI, *Graphical User Interface*) es el conjunto de elementos visuales —ventanas, botones, campos de texto, menús— que permiten al usuario interactuar con el software sin necesidad de escribir comandos.

Event Loop

El event loop es el núcleo de toda GUI: un bucle que espera eventos del SO (clics, teclado, timer), los despacha al manejador registrado y actualiza la pantalla. Bloquear el hilo del event loop congela la interfaz. Cualquier operación lenta (red, disco, cálculo pesado) debe ejecutarse en un hilo secundario.

13.1 Frameworks por Lenguaje

Framework	Lenguaje	Plataformas	Motor
Tkinter	Python	Win/Mac/Linux	Tcl/Tk
PyQt5/6	Python	Win/Mac/Linux	Qt C++
Electron	JS/TS	Win/Mac/Linux	Chromium+Node
Flutter	Dart	Todas (6 plat.)	Skia/Impeller
Swing	Java	Win/Mac/Linux	JVM
JavaFX	Java	Win/Mac/Linux	CSS + FXML
GTK	C/Python/Rust	Linux/Win/Mac	GTK4

13.2 Tkinter — Python

Tkinter está incluido en la instalación estándar de Python y no requiere dependencias adicionales. Es ideal para herramientas internas, scripts con UI y prototipos rápidos.

```

1 import tkinter as tk
2 from tkinter import ttk, messagebox, filedialog
3
4 class EditorNotas(tk.Tk):
5     def __init__(self):
6         super().__init__()
7         self.title("Editor de Notas")
8         self.geometry("860x580")
9         self._ruta = None

```

```

10         self._modificado = False
11         self._construir_ui()
12         self.bind("<Control-s>", lambda _: self._guardar())
13
14     def _construir_ui(self):
15         marco = tk.Frame(self)
16         marco.pack(fill=tk.BOTH, expand=True, padx=6, pady=6)
17         scroll = ttk.Scrollbar(marco)
18         scroll.pack(side=tk.RIGHT, fill=tk.Y)
19         self._txt = tk.Text(marco, font=("Courier New", 12),
20                             wrap=tk.WORD, undo=True,
21                             yscrollcommand=scroll.set)
22         self._txt.pack(fill=tk.BOTH, expand=True)
23         scroll.config(command=self._txt.yview)
24
25     def _guardar(self):
26         if self._ruta:
27             with open(self._ruta, "w", encoding="utf-8") as f:
28                 f.write(self._txt.get("1.0", tk.END))
29
30 if __name__ == "__main__":
31     EditorNotas().mainloop()

```

Listing 13.1: Tkinter — editor de notas simplificado

13.3 PyQt5 — Python

PyQt5 envuelve el framework C++ Qt, ofreciendo apariencia nativa, diseñador visual (Qt Designer) y soporte completo para multimedia, bases de datos y redes.

Señales y Slots

El mecanismo de comunicación de Qt. Una señal se emite cuando ocurre un evento (clic, cambio de valor). Un slot es cualquier función Python conectada a esa señal. La conexión es segura en tipos y puede cruzar hilos.

13.4 Electron — JavaScript

Electron permite construir aplicaciones de escritorio con tecnologías web. Combina Chromium (para la UI) y Node.js (para acceder al sistema). VS Code, Slack y Discord usan Electron.

13.5 Flutter — Dart

Flutter (Google, 2018) usa Dart y dibuja cada píxel con su propio motor gráfico (Skia/Impeller), logrando apariencia consistente en iOS, Android, Web, Windows, macOS y Linux desde un único código fuente.

13.6 Swing y JavaFX — Java

Swing y JavaFX son los frameworks GUI nativos de Java. Swing usa componentes basados en la JVM con apariencia personalizable; JavaFX ofrece FXML para separar vista de lógica y soporte para CSS.

13.7 Conceptos Esenciales de Diseño GUI

13.7.1 Principios de Usabilidad

- **Consistencia:** Los controles se comportan igual en toda la aplicación.
- **Retroalimentación:** Toda acción produce una respuesta visible (spinner, mensaje, cambio de color).
- **Prevención de errores:** Deshabilitar controles irrelevantes, validar antes de ejecutar.
- **Accesibilidad:** Contraste mínimo 4.5:1, navegación por teclado, etiquetas descriptivas.

13.7.2 Hilo Principal y Hilos Secundarios

Toda operación lenta debe ejecutarse en un hilo secundario y comunicar el resultado al hilo de la UI:

Framework	Hilo UI	Hilo secundario	Comunicación
Tkinter	mainloop	threading.Thread	after() callback
PyQt5/6	QApplication	QThread	pyqtSignal
Swing	EDT	SwingWorker	invokeLater()
Flutter	Isolate principal	compute()	StreamController
Electron	Renderer process	Main process	IPC invoke/handle

Referencias

- Cormen, T., Leiserson, C., Rivest, R., Stein, C. *Introduction to Algorithms*, 4th ed. MIT Press, 2022.
- Kernighan, B., Ritchie, D. *The C Programming Language*, 2nd ed. Prentice Hall, 1988.
- Stroustrup, B. *A Tour of C++*, 3rd ed. Addison-Wesley, 2022.
- Klabnik, S., Nichols, C. *The Rust Programming Language*. No Starch Press, 2022.
- Donovan, A., Kernighan, B. *The Go Programming Language*. Addison-Wesley, 2015.
- Pilgrim, M. *Dive Into Python 3*. Apress, 2009.
- Documentación oficial de Python: <https://docs.python.org>
- Documentación oficial de Qt: <https://doc.qt.io>
- Documentación oficial de Flutter: <https://flutter.dev/docs>
- Documentación oficial de Electron: <https://electronjs.org/docs>